

Problem 1 (55 points)

In this problem, you will use various classification algorithms to analyze a dataset of SMS messages, distinguishing between 'spam' and 'ham' (non-spam). Through this analysis, you will gain insights into how effectively different models can classify and what model characteristics lead to the best performance.

Consider the dataset file `'Spam_SMS.csv'`, which contains:

- Class: Labels the message as 'spam' or 'ham'.
- Message: Contains the text of the SMS.

This dataset is part of the SMS Spam Collection, a public set intended for mobile phone spam research. Collected from various free sources on the internet, it provides diverse examples for training classification models.

Please follow these steps:

(a) Data Preparation:

- Load the dataset `'Spam_SMS.csv'` using pandas' `'read_csv()'` function.
- Convert the 'Class' labels in the dataset from 'spam' and 'ham' to binary values 1 and 0, respectively.
- Split the dataset into training and testing sets using an 80/20 ratio with the `'train_test_split()'` function from scikit-learn. Use a `random_state` of 42 to ensure reproducibility.
- Perform Term Frequency-Inverse Document Frequency (TF-IDF) vectorization on the text data using `'TfidfVectorizer'` from scikit-learn. Exclude English stop words during this process. Note that you need to first fit the vectorizer on the training data to learn the vocabulary and inverse document frequency, and then transform both the training and testing sets into numerical features based on this learned information.

(b) Naive Bayes Classifier:

- Fit a Naive Bayes model using the `'MultinomialNB'` classifier from scikit-learn on the TF-IDF-transformed training data. Use the default parameters.
- Compute and display the accuracy, sensitivity, and specificity of the model on the testing data.
- Display the confusion matrix to visualize the model's performance.

(c) k-Nearest Neighbors (KNN) Classifier:

- Fit a k-Nearest Neighbors model using the `'KNeighborsClassifier'` from scikit-learn on the TF-IDF-transformed training data. Use the default parameters first.

- Explain what the ``n_neighbors`` parameter is and identify its default value.
- Tune the ``n_neighbors`` parameter. Present the test accuracy for each value. Select the best model based on test accuracy.
- For the best model selected, compute and display the accuracy, sensitivity, and specificity of the model on the testing data.
- Display the confusion matrix to visualize the model's performance.

(d) Logistic Regression:

- Fit a Logistic Regression model using the ``LogisticRegression`` classifier from scikit-learn on the TF-IDF-transformed training data. Use the default parameters.
- Compute and display the accuracy, sensitivity, and specificity of the model on the testing data.
- Display the confusion matrix to visualize the model's performance.

(e) Support Vector Machine (SVM):

- Fit an SVM using the ``SVC`` model from scikit-learn on the TF-IDF-transformed training data. Start with the default parameters.
- Explain what the ``kernel`` parameter is and identify its default value. If its value changes from ``linear`` to ``poly``, how does it affect the decision boundary? Be specific.
- Explain what the ``C`` parameter is and identify its default value. If ``C`` increases, how does it affect the margin and the number of support vectors? Provide details.
- Tune the ``kernel`` and ``C`` parameters. Present the test accuracy for each combination. Select the best model based on test accuracy.
- For the best model selected, compute and display the accuracy, sensitivity, and specificity on the testing data.
- Display the confusion matrix to visualize the model's performance.

(f) Random Forest:

- Fit a Random Forest model using the ``RandomForestClassifier`` from scikit-learn on the TF-IDF-transformed training data. Start with the default parameters.
- Explain what the ``n_estimators`` parameter is and identify its default value. Discuss how increasing ``n_estimators`` impacts model performance and variance.
- Explain what the ``max_depth`` parameter is. Note that the default is ``max_depth=None``. Explain what this setting means.
- Explain what the ``max_features`` parameter is and how it affects the diversity of the trees. If ``max_features`` increases, will the trees become more or less diverse?
- For the chosen parameter settings, compute and display the accuracy, sensitivity, and specificity on the testing data.
- Display the confusion matrix to visualize the model's performance.

(g) Plotting Model Performance:

- Sort the models by test accuracy in descending order. Create a bar chart to display the sorted test accuracy for each model, ensuring the plot is clearly labeled and titled accordingly. Based on this accuracy bar chart, select the best-performing model.
- Create a scatterplot to visualize test sensitivity against test specificity for all models in one chart. Ensure the plot is clearly labeled and includes annotations for each model. Discuss the trade-off between sensitivity and specificity.

SOLUTION:

--- (Student's Solution Here) ---

Problem 2 (10 points)

In this problem, you will build a neural network to classify SMS messages using the same dataset and data splitting approach as in Problem 1. However, you may apply any other preprocessing and feature creation methods.

Follow these steps:

- Preprocess the data and create features suitable for training a neural network of your choice.
- Train the neural network. Evaluate its performance on the testing data.
- Compute and display the test accuracy, test sensitivity, and test specificity of the model.
- Recreate the two charts from Problem 1 here, including the neural network's performance. Discuss any differences in the results and potential reasons.

A neural network, with the appropriate architecture, is expected to outperform other models on this dataset. To receive full credit, design a neural network that achieves a test accuracy of at least 0.98.

SOLUTION:

--- (Student's Solution Here) ---